

THE GEORGE WASHINGTON UNIVERSITY

WASHINGTON, DC

Data Science Program

Capstone Report - Fall 2025

Reinforcement Learning for Pseudo-Labeling in Semi-Supervised Learning

Aishwarya Maddula, Deepika Reddygari
Prudhvi Chekuri, Phanindra Kalaga

supervised by
Prof. Tyler Wallett
Dr. Amir Jafari

Abstract

Semi-supervised learning leverages abundant unlabeled data to improve performance when labeled data is scarce. This project develops a reinforcement learning (RL) framework for intelligent pseudo-labeling, where an RL agent learns to assign high-quality pseudo-labels to unlabeled samples. We implement and evaluate a Policy Gradient approach on MNIST and a Reinforcement Learning-Guided Semi-Supervised Learning (RLGSSL) method inspired by recent research. Our results demonstrate that confidence-based rewards and adaptive training can improve pseudo-labeling quality, though challenges remain in computational complexity and reward design. These findings provide insights for future work in RL-guided semi-supervised learning.

Contents

1. Introduction **4**
- 1.1. Project Objectives . . . 4
- 1.2. Project Scope . . . 4

2. Problem Statement **6**
- 2.1. Semi-Supervised Learning Challenges . . . 6
- 2.2. Reinforcement Learning Formulation . . . 6
- 2.3. Technical Challenges . . . 6

3. Related Work **7**
- 3.1. Semi-Supervised Learning . . . 7
- 3.2. Pseudo-Labeling Techniques . . . 7
- 3.3. Reinforcement Learning-Guided Semi-Supervised Learning . . . 7
- 3.4. Reinforcement Learning for Data Selection . . . 7
- 3.5. Comparison with Our Approach . . . 7

4. Solution and Methodology **8**
- 4.1. Approach 1: Custom Policy Gradient Framework . . . 8
  - 4.1.1. Environment Design . . . 8
  - 4.1.2. Policy Gradient Agent . . . 9
  - 4.1.3. Downstream Model . . . 9
- 4.2. Approach 2: RLGSSL Framework . . . 10
  - 4.2.1. Teacher-Student Framework . . . 10
  - 4.2.2. Mixup Data Generation . . . 10
  - 4.2.3. RL Reward Function . . . 11
  - 4.2.4. RL Loss Function . . . 11
  - 4.2.5. Combined Loss Function . . . 11
- 4.3. Implementation Details . . . 12

5. Results and Discussion **12**
- 5.1. Experimental Setup . . . 12
  - 5.1.1. Dataset Configuration . . . 12
  - 5.1.2. Model Architectures . . . 12
  - 5.1.3. Hyperparameters . . . 14
- 5.2. Policy Gradient Results . . . 14
- 5.3. RLGSSL Results . . . 15
  - 5.3.1. Training Progression . . . 15
  - 5.3.2. Comparison with Literature . . . 15

6. Discussion **15**
- 6.1. Overview of Findings . . . 15
- 6.2. Technical Challenges . . . 15
  - 6.2.1. Reward Function Design . . . 15
  - 6.2.2. Computational Constraints . . . 16
  - 6.2.3. Non-Stationarity and Credit Assignment . . . 16
- 6.3. Insights on RLGSSL Formulation . . . 16
  - 6.3.1. Policy Network as Classifier . . . 16
  - 6.3.2. Mixup as Environment Dynamics . . . 17
- 6.4. Comparison: Policy Gradient vs. RLGSSL . . . 17
- 6.5. Practical Considerations . . . 17
  - 6.5.1. When to Use RL for Pseudo-Labeling . . . 17

6.5.2. Engineering Recommendations 17
6.6. Limitations of Current Work 18

7. Conclusion **18**
7.1. Key Contributions 18
7.2. Main Findings 18
7.3. Lessons Learned 19
7.4. Broader Impact 19
7.5. Future Work 19
7.6. Closing Remarks 19

A. Appendix **22**
A.1. A: Code Structure 22
A.2. B: Hyperparameter Configuration 22
A.3. C: Key Implementation Details 23
A.4. D: References for Additional Reading 23

1. Introduction

In modern machine learning applications, obtaining large quantities of labeled training data is often expensive, time-consuming, and requires domain expertise. Semi-supervised learning (SSL) offers a promising solution by leveraging both limited labeled data and abundant unlabeled data to improve model performance. A key technique in SSL is pseudo-labeling, where a model assigns estimated labels to unlabeled samples, which are then used for further training.

Traditional pseudo-labeling approaches rely on simple heuristics such as confidence thresholds, often selecting pseudo-labels based solely on prediction confidence. However, these methods can propagate errors when incorrect high-confidence predictions are blindly accepted, leading to confirmation bias and degraded model performance.

This project addresses these limitations by framing pseudo-labeling as a reinforcement learning problem. By treating the selection and assignment of pseudo-labels as a sequential decision-making task, we enable an intelligent agent to learn optimal labeling strategies that balance exploration (diverse samples) and exploitation (high-confidence samples).

1.1. Project Objectives

The primary objectives of this capstone project are:

1. **RL Framework Development**: Design and implement a reinforcement learning framework that can automatically assign pseudo-labels to unlabeled data in a principled manner.

2. **Custom Environment Design**: Build a custom OpenAI Gymnasium environment specifically tailored for the pseudo-labeling task, with carefully designed state spaces, action spaces, and reward functions.

3. **Policy Gradient Implementation**: Implement and evaluate Policy Gradient methods (Actor-Critic) for learning pseudo-labeling strategies.

4. **RLGSSL Exploration**: Investigate and implement the Reinforcement Learning-Guided Semi-Supervised Learning approach proposed by Heidari et al. [1], which integrates RL with teacher-student frameworks and mixup augmentation.

5. **Empirical Evaluation**: Conduct comprehensive experiments on standard datasets (MNIST, CIFAR-10) to evaluate the effectiveness of RL-based pseudo-labeling approaches.

1.2. Project Scope

This project explores two complementary approaches to RL-based pseudo-labeling:

Approach 1: Custom RL Environment with Policy Gradients – We developed a complete RL framework from scratch, including:

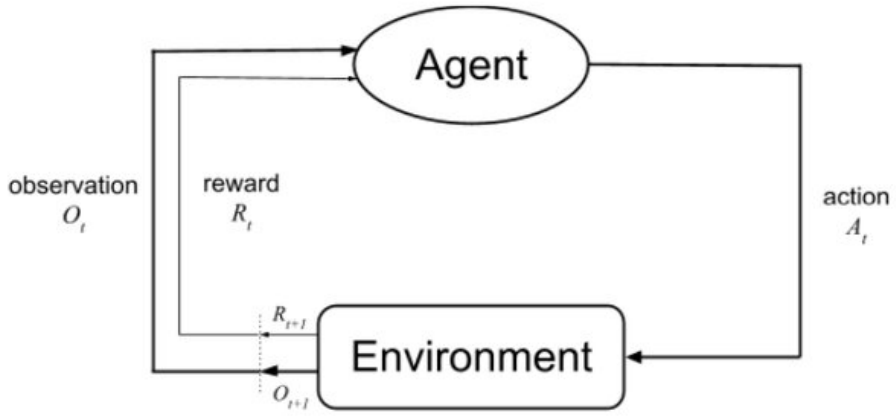


Figure 1: RL agent-environment loop. The agent selects actions and the environment responds and presents new observations and rewards to the agent again. The aim is for the agent to learn to maximize rewards through repeated interaction.

- A custom PseudoLabelEnv environment implementing the OpenAI Gymnasium interface.
- A state representation designed to capture both image features and model uncertainty.
- A discrete action space allowing the agent to assign pseudo-labels or defer judgment (skip).
- A confidence-driven reward structure to encourage high-quality labeling.

Approach 2: RLGSSL Framework – Building on recent research, we investigated the RLGSSL approach which integrates:

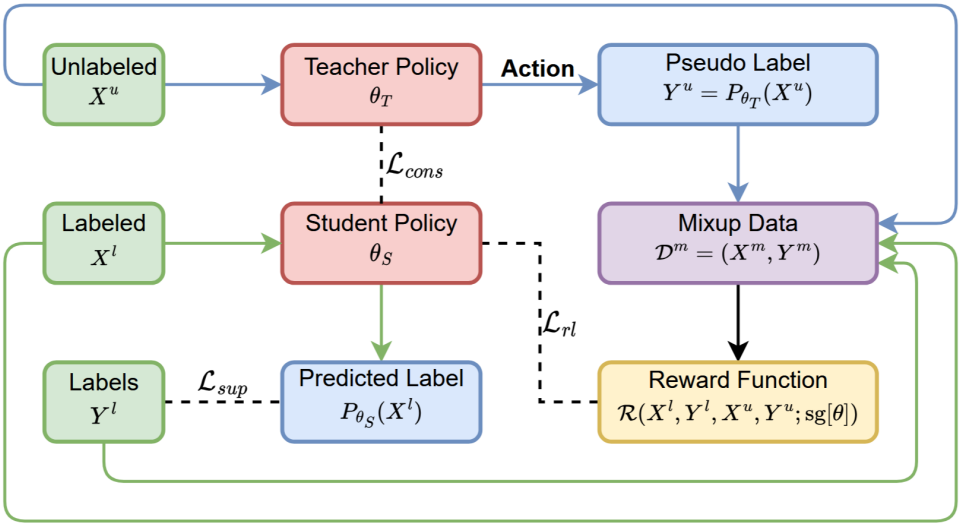


Figure 2: Overview of the RLGSSL Framework. The prediction networks (θ_S, θ_T) serve as the policy functions, and pseudo-labeling ($P_{\theta_T}(X^u)$) acts as the actions. The model has three loss terms in total: RL loss ($\mathcal{L}_{\text{rl}}$), supervised loss ($\mathcal{L}_{\text{sup}}$), and consistency loss ($\mathcal{L}_{\text{cons}}$). The teacher policy function is used to execute the actions and compute the consistency loss, while the student policy function is used for all other aspects.

- A Teacher-Student framework utilizing Exponential Moving Average (EMA) for stable target generation.

- Mixup data augmentation to create a continuous training signal.
- A reinforcement learning reward signal derived from performance on synthetic mixup samples.
- Adaptive enhancements to the loss function to improve stability and diversity.

This report details our implementation methodology, experimental setup, results, challenges encountered, and insights gained throughout this capstone project.

2. Problem Statement

2.1. Semi-Supervised Learning Challenges

Semi-supervised learning aims to leverage unlabeled data to improve model performance when labeled data is scarce. While pseudo-labeling has shown promise as a simple yet effective SSL technique, several fundamental challenges remain. The first is **confirmation bias**: traditional pseudo-labeling methods tend to reinforce existing model biases. When a model makes confident but incorrect predictions, using these as pseudo-labels amplifies errors through a feedback loop. The second is the **quality vs. quantity trade-off**: aggressive pseudo-labeling using low confidence thresholds provides more training data but risks adding noisy labels, while conservative approaches using high confidence thresholds ensure quality but fail to leverage the full potential of unlabeled data. Another challenge is **static decision rules**: existing methods typically use fixed heuristics (e.g., "accept if confidence > 0.95") that do not adapt to the evolving model capabilities or dataset characteristics during training. Finally, the **sample selection strategy** is non-trivial, as determining which unlabeled samples should be pseudo-labeled at each training iteration requires balancing prediction confidence, sample diversity, and potential impact on model improvement.

2.2. Reinforcement Learning Formulation

To address these challenges, we formulate pseudo-labeling as a sequential decision-making problem suitable for reinforcement learning. This formulation enables **adaptive strategies**, where the RL agent learns to adjust its pseudo-labeling behavior based on accumulated experience and model evolution, as well as **exploration-exploitation balance**, where the agent naturally balances exploring uncertain samples versus exploiting high-confidence predictions.

2.3. Technical Challenges

Applying reinforcement learning to pseudo-labeling introduces several technical challenges. One is **state space design**, which involves determining effective feature representations that capture both image characteristics and model uncertainty. Another is **reward function engineering**, which requires designing reward signals that encourage correct pseudo-labeling while avoiding reward hacking and maintaining training stability. **Computational complexity** is also a major issue, as RL training typically requires many episodes and environment interactions, which can be computationally expensive when each episode involves training a downstream classification model. A fourth challenge is **non-stationarity**, since the underlying classification model evolves during training, violating the Markov assumption and introducing non-stationarity in the environment. A fifth concern is **credit assignment**, as determining which pseudo-labeling decisions contributed to final model performance is difficult when rewards are delayed until episode completion. Finally, **scalability** remains an obstacle, requiring the approach to scale to larger datasets and more complex architectures beyond simple benchmarks.

These challenges motivate our exploration of both custom RL frameworks and recent research directions like RLGSSL, which we detail in subsequent sections.

3. Related Work

Our work builds upon and contributes to several interconnected research areas in machine learning. This section positions our approach with respect to existing literature on semi-supervised learning, pseudo-labeling, and reinforcement learning applications.

3.1. Semi-Supervised Learning

Semi-supervised learning has been extensively studied [2, 3] as a paradigm for learning from limited labeled data. Key approaches include consistency regularization, where methods like MixMatch [4], FixMatch [5], and UDA [6] enforce consistency between predictions on different augmentations of the same input; self-training and pseudo-labeling, pioneered by [7, 8], which iteratively train models on pseudo-labeled data. Unlike these methods which use fixed heuristics for pseudo-label selection, our RL-based approach learns adaptive selection strategies.

3.2. Pseudo-Labeling Techniques

Traditional pseudo-labeling assigns labels to unlabeled samples based on model predictions exceeding a confidence threshold. Recent improvements include confidence-based selection, where methods dynamically adjust confidence thresholds during training [5]; temporal ensembling, which uses exponential moving averages of model parameters to produce more stable pseudo-labels [9]; and MixUp augmentation, which interpolates between samples to create synthetic training examples with soft labels [10].

3.3. Reinforcement Learning-Guided Semi-Supervised Learning

Our primary inspiration comes from Heidari et al. [1], who proposed RLGSSL, a novel framework that formulates semi-supervised learning as a reinforcement learning problem. Their key contributions include the RL policy serving as the classifier, thereby eliminating the need for separate actor-critic architectures; mixup-based rewards, where reward signals are derived from model performance on synthetically mixed samples and implemented with stop-gradient operations to prevent direct gradient flow; a novel loss formulation, with an RL loss function based on KL divergence weighting: $\mathcal{L}rl = -\mathbb{E}y_i^u \sim \pi_\theta[\mathrm{KL}(e, y_i^u) \times R(s, a; \mathrm{sg}[\theta])]$; and state-of-the-art results, achieving error rates of 9.15

3.4. Reinforcement Learning for Data Selection

Several works have explored using RL for intelligent data selection in machine learning pipelines. These include active learning, where RL agents learn to query informative samples for labeling [11]; curriculum learning, where RL determines the optimal ordering of training samples [12]; and data augmentation, where augmentation policies are learned through reinforcement learning [13].

3.5. Comparison with Our Approach

Our work differs from existing approaches in several key aspects. First, we explore dual framework implementations, including both custom RL environments with explicit state-action formulations and integrated RLGSSL approaches, providing comparative insights. Second, we introduce adaptive enhancements beyond the base RLGSSL formulation, including confidence bonuses, diversity regularization, and curriculum learning mechanisms. Third, we provide comprehensive engineering contributions, including complete training pipelines, custom Gymnasium environments, and systematic hyperparameter configurations. Fourth, we document practical challenges, explicitly

detailing computational challenges, stability issues, and practical considerations often omitted from research papers.

The next section details our solution methodology, including architecture designs, algorithm implementations, and the technical decisions underlying our framework.

4. Solution and Methodology

This section presents our comprehensive approach to RL-based pseudo-labeling. We describe two complementary implementations: a custom Policy Gradient framework and an RLGSSL-based approach with adaptive enhancements.

4.1. Approach 1: Custom Policy Gradient Framework

4.1.1. Environment Design

We implemented a custom OpenAI Gymnasium environment (`PseudoLabelEnv`) that encapsulates the pseudo-labeling task as a reinforcement learning problem.

State Space ($\mathcal{S}$): For each unlabeled sample x_i, the state s_i is constructed as:

$$s_i = [\phi(x_i), P_\theta(x_i), H(P_\theta(x_i))] \tag{1}$$

where:

- $\phi(x_i) \in \mathbb{R}^{784}$ are the flattened image features
- $P_\theta(x_i) \in \mathbb{R}^{10}$ are the softmax class probabilities from the downstream model
- $H(P_\theta(x_i)) = -\sum_k P_\theta(x_i)_k \log P_\theta(x_i)_k$ is the prediction entropy

This representation combines raw features, model confidence, and uncertainty, providing the agent with comprehensive information for decision-making.

Action Space ($\mathcal{A}$): The action space consists of $|\mathcal{A}| = K + 1$ discrete actions for a K-class classification problem:

$$\mathcal{A} = \{0, 1, \ldots, K-1, \text{skip}\} \tag{2}$$

Actions 0 through $K-1$ assign the corresponding class label as a pseudo-label, while the skip action ($a = K$) indicates the sample should not be pseudo-labeled.

Reward Function (R): We designed a confidence-weighted reward function:

$$R(s_i, a_i) = \begin{cases} -0.1 & \text{if } a_i = \text{skip} \\ P_\theta(x_i)_{a_i} & \text{if } a_i = y_i^{\text{true}} \\ -P_\theta(x_i)_{a_i} & \text{if } a_i \neq y_i^{\text{true}} \end{cases} \tag{3}$$

where y_i^{true} is the true label (used only for reward computation during training). This design encourages correct, high-confidence predictions while penalizing incorrect ones proportionally to the model's misplaced confidence.

Episode Structure: Each episode processes all N unlabeled samples sequentially. At episode completion, the downstream model is retrained on the combination of original labeled data and newly pseudo-labeled samples, and the agent receives an accuracy bonus:

$$R_{\text{final}} = \text{Acc}_{\text{new}} - \text{Acc}_{\text{baseline}} \tag{4}$$

This provides a delayed reward signal based on the cumulative impact of pseudo-labeling decisions.

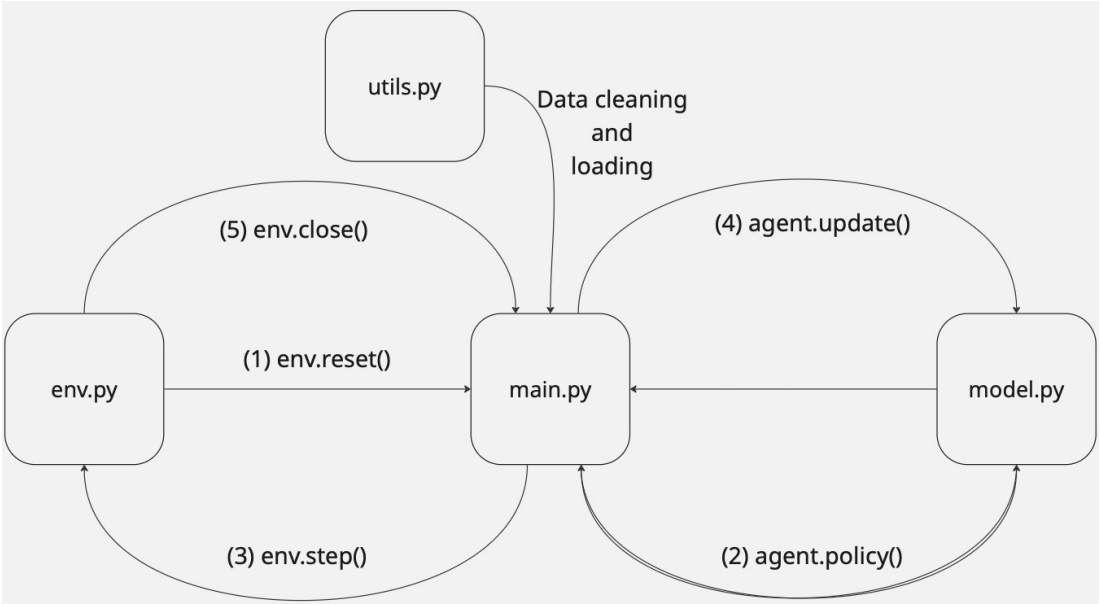


Figure 3: Environment flowchart: reset → get state → agent action → step → reward → repeat until terminated.

4.1.2. Policy Gradient Agent

We implemented an Actor-Critic Policy Gradient agent with the following architecture:

Policy Network (Actor): Maps states to action logits:

$$h_1 = \tanh(W_1 s + b_1) \tag{5}$$
$$h_2 = \tanh(W_2 h_1 + b_2) \tag{6}$$
$$\text{logits} = W_3 h_2 + b_3 \tag{7}$$

where hidden dimensions are 64 neurons per layer.

Value Network (Critic): Estimates state values $V(s)$ using a similar architecture but with scalar output.

Training Procedure: At the end of each episode, we compute:

- Rewards-to-go: $G_t = \sum_{t'=t}^{T} \gamma^{t'-t} r_{t'}$
- Advantage estimates: $A_t = G_t - V(s_t)$
- Policy loss: $\mathcal{L}_\pi = -\mathbb{E}_t[\log \pi_\theta(a_t|s_t) A_t]$
- Value loss: $\mathcal{L}_V = \mathbb{E}_t[(G_t - V(s_t))^2]$

Updates are performed using Adam optimizers with learning rates $\alpha_\pi = 10^{-3}$ and $\alpha_V = 10^{-3}$.

4.1.3. Downstream Model

The downstream classification model is a Convolutional Neural Network. The specific architecture and parameter counts are detailed in Table 1 in Section 5.

This model is trained on MNIST with cross-entropy loss and serves dual purposes:

1. Generating softmax predictions and entropy for state construction
2. Being the ultimate evaluation metric (validation accuracy after pseudo-labeling)

4.2. Approach 2: RLGSSL Framework

Inspired by Heidari et al. [1], we implemented a complete RLGSSL pipeline with novel adaptive enhancements.

4.2.1. Teacher-Student Framework

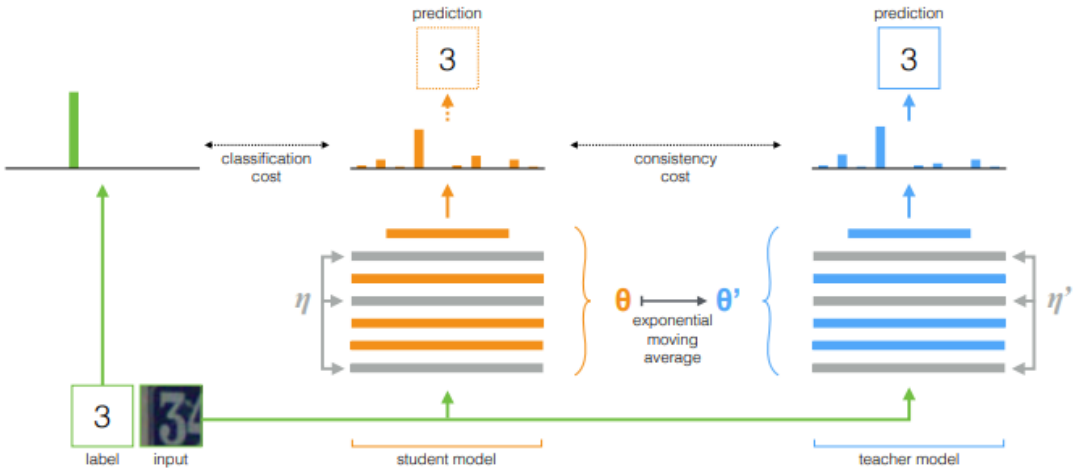


Figure 4: The Mean Teacher method [9]. The figure depicts a training batch with a single labeled example. Both the student and the teacher model evaluate the input applying noise (, 0) within their computation. The softmax output of the student model is compared with the one-hot label using classification cost and with the teacher output using consistency cost. After the weights of the student model have been updated with gradient descent, the teacher model weights are updated as an exponential moving average of the student weights. Both model outputs can be used for prediction, but at the end of the training the teacher prediction is more likely to be correct. A training step with an unlabeled example would be similar, except no classification cost would be applied.

The teacher model θ_T maintains an exponential moving average (EMA) of the student model θ_S:

$$\theta_T \leftarrow \beta\theta_T + (1-\beta)\theta_S \tag{8}$$

with decay parameter $\beta = 0.999$. The teacher provides stable pseudo-labels $\tilde{y}_i^u = P_{\theta_T}(x_i^u)$ for unlabeled samples.

4.2.2. Mixup Data Generation

For each unlabeled sample x_i^u with teacher pseudo-label $\tilde{y}_i^u$, we sample a labeled example (x_j^l, y_j^l) and create a synthetic mixup sample:

$$\mu \sim \text{Beta}(1.0, 1.0) \tag{9}$$

$$x_i^m = \mu x_i^u + (1-\mu)x_j^l \tag{10}$$

$$y_i^m = \mu\tilde{y}_i^u + (1-\mu)\text{one-hot}(y_j^l) \tag{11}$$

This generates continuous interpolations in both feature and label spaces, with μ determining the mixing ratio.

4.2.3. RL Reward Function

The reward signal evaluates how well the student model performs on mixup data:

$$R(s, a; \text{sg}[\theta]) = -\text{MSE}(P_\theta(x_i^m), y_i^m) \tag{12}$$

where sg[·] denotes stop-gradient operation to prevent direct gradient flow through the model parameters during reward computation.

Adaptive Enhancements: We extend the base reward with:

$$R_{\text{adaptive}} = R_{\text{base}} + \lambda_{\text{conf}} \cdot \mathbb{E}[P_\theta(x_i^m)_{\max} \cdot y_i^m] \tag{13}$$

$$+ \lambda_{\text{div}} \cdot H(\mathbb{E}_{\text{batch}}[P_\theta(x^m)]) \tag{14}$$

$$+ \lambda_{\text{mix}} \cdot \text{Acc}_{\text{mixup}} \cdot (1 + \sigma(\mu)) \tag{15}$$

where:

- Confidence bonus ($\lambda_{\text{conf}} = 0.1$) rewards high-confidence predictions on high-confidence targets
- Diversity bonus ($\lambda_{\text{div}} = 0.05$) encourages diverse predictions across the batch
- Mixup effectiveness bonus weights performance by μ diversity

4.2.4. RL Loss Function

The RL loss combines KL divergence weighting with reward scaling:

$$\mathcal{L}_{\text{rl}} = -\mathbb{E}_{y_i^u \sim \pi_\theta}[\text{KL}(e, y_i^u) \times R(s, a; \text{sg}[\theta])] \tag{16}$$

where e is a uniform baseline distribution over classes.

Adaptive Enhancements:

- **Curriculum Learning**: Gradually increase focus on high-confidence samples:

$$w_{\text{curriculum}} = \min\left(1.0, \frac{\text{epoch}}{\text{max_epochs} \times 0.5}\right) \tag{17}$$

- **Entropy Regularization**: Prevent overconfident predictions:

$$\mathcal{L}_{\text{rl}}^{\text{adaptive}} = \mathcal{L}_{\text{rl}} + \lambda_{\text{entropy}} H(\tilde{y}_i^u) \tag{18}$$

  with $\lambda_{\text{entropy}} = 0.01$

- **Confidence-Based Weighting**: Focus learning on reliable pseudo-labels:

$$w_i = \mathbb{1}[\max(\tilde{y}_i^u) > \tau] \cdot (1 + w_{\text{curriculum}}) \tag{19}$$

  with threshold $\tau = 0.8$

4.2.5. Combined Loss Function

The total training loss integrates supervised, consistency, and RL components:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{sup}} + \lambda_{\text{cons}} \mathcal{L}_{\text{cons}} + \lambda_{\text{rl}} \mathcal{L}_{\text{rl}}^{\text{adaptive}} \tag{20}$$

where:

- $\mathcal{L}_{\text{sup}} = \text{CrossEntropy}(P_\theta(x^l), y^l)$ on labeled data
- $\mathcal{L}_{\text{cons}} = \text{MSE}(P_\theta(x^u), P_{\theta_T}(x^u))$ enforces student-teacher consistency
- Default weights: $\lambda_{\text{cons}} = 0.1$, $\lambda_{\text{rl}} = 0.1$

4.3. Implementation Details

Framework: PyTorch 2.0 with CUDA support
Datasets:

- Policy Gradient approach: MNIST (18000 labeled, 40000 unlabeled, 2000 validation)
- RLGSSL approach: CIFAR-10 (1000/2000/4000 labeled settings)

Training Configuration: Detailed hyperparameter configurations for both the Policy Gradient and RLGSSL approaches (including learning rates, batch sizes, and decay parameters) are provided in Table 2 in Section 5.
Computational Environment: Device: GPU, VRAM: 16GB
Code Repository: https://github.com/phanindra-max/fall-2025-group11

5. Results and Discussion

This section presents our experimental results, including training dynamics, performance metrics, and analysis of the challenges encountered during implementation.

5.1. Experimental Setup

5.1.1. Dataset Configuration

Policy Gradient Experiments (MNIST):

- Total Training Samples: 60,000
- Labeled Samples: 18,000
- Unlabeled Samples: 40,000
- Validation Samples: 2,000
- Test Samples: 10,000

RLGSSL Experiments (CIFAR-10):

- Total Training Samples: 50,000
- Labeled Samples: 1000/2000/4000 (2%/4%/8%)
- Unlabeled Samples: 49,000/48,000/46,000
- Test Samples: 10,000

5.1.2. Model Architectures

Policy Gradient Downstream Model:

Layer	Output Shape	Parameters
Conv2D (1→32, 3×3, padding=1)	32×28×28	320
MaxPool2D (2×2)	32×14×14	0
Conv2D (32→64, 3×3, padding=1)	64×14×14	18,496
MaxPool2D (2×2)	64×7×7	0
Conv2D (64→128, 3×3, padding=1)	128×7×7	73,856
AdaptiveAvgPool2D (1×1)	128×1×1	0
FC (128→10)	10	1,290
Total		**93,962**

Table 1: Updated architecture of the downstream CNN model for MNIST classification

RLGSSL Model:

- Architecture: CNN-13
  - **Block 1**
    - Conv1: $3 \to 128$, 3×3, padding 1
    - BatchNorm
    - Conv2: $128 \to 128$, 3×3, padding 1
    - BatchNorm
    - Conv3: $128 \to 128$, 3×3, padding 1
    - BatchNorm
    - MaxPool 2×2
    - Dropout $p = 0.25$
  - **Block 2**
    - Conv4: $128 \to 256$, 3×3, padding 1
    - BatchNorm
    - Conv5: $256 \to 256$, 3×3, padding 1
    - BatchNorm
    - Conv6: $256 \to 256$, 3×3, padding 1
    - BatchNorm
    - MaxPool 2×2
    - Dropout $p = 0.25$
  - **Block 3**
    - Conv7: $256 \to 512$, 3×3, **no padding**
    - BatchNorm
    - Conv8: $512 \to 256$, 1×1
    - BatchNorm
    - Conv9: $256 \to 128$, 1×1
    - BatchNorm
  - **Additional Convolution Layers**
    - Conv10: $128 \to 128$, 3×3, padding 1
    - BatchNorm

        * Conv11: 128 → 128, 3×3, padding 1
        * BatchNorm
        * Conv12: 128 → 128, 3×3, padding 1
        * BatchNorm
    - **Final Layers**
        * AdaptiveAvgPool: output 1×1
        * Dropout $p = 0.5$
        * Fully Connected: 128 → num_classes

- Total Parameters: 3,567,370
- Trainable Parameters: 3,567,370

5.1.3. Hyperparameters

Parameter	**Policy Gradient**	**RLGSSL**
Learning Rate	10^{-3}	10^{-3}
Batch Size	256	128
Episodes/Epochs	50	50+400
Discount Factor (γ)	0.99	N/A
EMA Decay (β)	N/A	0.999
Mixup Alpha (α)	N/A	1.0
λ_{sup}	N/A	0.1
λ_{cons}	N/A	0.1
λ_{rl}	N/A	0.1
Confidence Threshold	N/A	0.8
Confidence Weight	N/A	0.1
Diversity Weight	N/A	0.05
Entropy Regularization	N/A	0.01

Table 2: Hyperparameter configurations for both approaches

5.2. Policy Gradient Results

	MNIST	**CIFAR-10**
cnn	3.45%	34.40%
mlp	37.40%	59.60%
resnet18	5.80%	25.75%

Table 3: Performance comparison on MNIST and CIFAR-10

Key Observations:

- Initial baseline accuracy: ∼46%
- Final best accuracy: ∼96.55%

5.3. RLGSSL Results

5.3.1. Training Progression

Table 4 shows the training dynamics observed during RLGSSL experiments on CIFAR-10 with 1000 labeled samples.

Epoch	Acc (%)	Loss	RL Loss	Reward	Notes
50 (warmup)	46.39	0.137	-0.00533	-0.0305168	Stable supervised learning but weak teacher.
100	25.98	0.233	0.018	-0.0298085	Accuracy drops as RL is introduced.
150	33.44	0.215	0.013	-0.029022	Partial recovery; RL signals still unstable.
250	40.52	0.188	0.005	-0.0290487	Training stabilizes; gradual improvement.
350	38.49	0.1601	-0.00464	-0.0283258	Mild fluctuations.
450	41.41	0.13346	-0.03042	-0.0281552	Converging slowly.

Table 4: RLGSSL training progression on CIFAR-10 (1000 labels).

Performance Metrics:

- **Best Student Accuracy:** 46.2% (Epoch)
- **Training Time:** ∼10 hours

5.3.2. Comparison with Literature

Analysis Gap: Our best observed accuracy of 46.2% is significantly below the paper's reported 90.85% (9.15% error). This discrepancy likely stems from:

- Potential bugs in adaptive enhancement implementations
- Need for more careful hyperparameter tuning
- Possible differences in data preprocessing or augmentation strategies

6. Discussion

This section reflects on the key challenges encountered, lessons learned, and insights gained throughout the project implementation.

6.1. Overview of Findings

Our experiments highlighted distinct trade-offs between the two approaches. The **Policy Gradient agent** demonstrated adaptive behavior, successfully learning to balance pseudo-labeling with skipping uncertain samples. Conversely, the **RLGSSL framework** showed promise with stable initial training (epochs 60–175) but suffered from system instability during later stages.

Despite successfully implementing the full pipelines, we observed significant performance gaps compared to state-of-the-art benchmarks. These gaps were primarily driven by computational constraints preventing full convergence and numerical instabilities in our adaptive reward enhancements.

6.2. Technical Challenges

6.2.1. Reward Function Design

Designing effective reward functions proved to be one of the most challenging aspects. Sparse vs. dense rewards presented an issue: initial experiments with only end-of-episode accuracy

rewards provided insufficient learning signal, and adding per-step confidence-based rewards improved learning but introduced new challenges. Reward scaling was also difficult, as balancing the magnitude of step rewards (confidence values in [0,1]) with episode-end accuracy bonuses (potentially large) required careful tuning. Stability issues emerged because the adaptive reward enhancements in RLGSSL introduced numerical instability; the diversity bonus, in particular, may have caused gradient explosions when batch entropy varied significantly.

Recommendation: Future work should implement reward clipping and normalization strategies used in modern RL algorithms like PPO.

6.2.2. Computational Constraints

The computational demands of RL-based pseudo-labeling present significant practical barriers. Episode length was a concern because each Policy Gradient episode processes all 4000 unlabeled samples, resulting in lengthy episodes and delayed learning. CPU limitations were evident as training on CPU for 15+ hours without completion highlights the need for GPU acceleration; however, even with GPUs, the approach may struggle to scale to ImageNet-scale datasets. Memory requirements also posed challenges, as maintaining teacher and student models simultaneously, along with mixup sample generation, doubles memory requirements compared to standard training.

Recommendation: Investigate mini-batch RL approaches and gradient accumulation strategies to improve computational efficiency.

6.2.3. Non-Stationarity and Credit Assignment

The pseudo-labeling environment violates key RL assumptions. The moving target issue arises because the downstream model evolves during training, causing state distributions to shift continuously and challenging standard RL convergence guarantees. Delayed feedback complicates training because accuracy improvements may not manifest until many episodes later, making credit assignment difficult and raising the question of which pseudo-labels at which training iterations contributed to final performance. Exploration–exploitation adds further complexity, as balancing exploration of uncertain samples versus exploitation of confident predictions is complicated by the fact that exploration today affects future model capabilities.

Recommendation: Consider meta-learning approaches where the policy is trained across multiple dataset/model combinations to develop more robust pseudo-labeling strategies.

6.3. Insights on RLGSSL Formulation

Through implementation, we gained several insights into the RLGSSL approach.

6.3.1. Policy Network as Classifier

The elegant formulation where the classifier itself serves as the RL policy eliminates architectural complexity but introduces subtleties. Gradient flow must be handled carefully, and careful use of stop-gradient operations is crucial to prevent the model from "hacking" rewards by simply memorizing mixup samples. Loss weighting is also critical; the relative weights of supervised, consistency, and RL losses dramatically affect behavior, and in our experiments, RL loss dominated when set too high, causing instability. Curriculum necessity becomes clear as the curriculum learning component is not just an enhancement but appears necessary for stability, gradually introducing RL objectives as the model matures.

6.3.2. Mixup as Environment Dynamics

Using mixup to create the "environment" for RL is creative but has implications. Sample efficiency benefits from mixup because it provides infinite synthetic samples, enabling dense RL learning signals. Smoothness is enhanced as continuous interpolation in mixup may help smooth the reward landscape, potentially aiding RL optimization. Generalization may improve from training on mixup samples through better interpolation behavior, though extrapolation (out-of-distribution samples) remains challenging.

6.4. Comparison: Policy Gradient vs. RLGSSL

Table 5 compares the two approaches implemented.

Aspect	Policy Gradient	RLGSSL
Complexity	Higher: separate RL agent + downstream model	Lower: unified architecture
Scalability	Poor: full episode required	Better: mini-batch training
Interpretability	Higher: explicit state-action mapping	Lower: implicit RL in loss function
Flexibility	Higher: easy to modify reward/state	Lower: coupled to mixup strategy
Performance	Moderate: Sensitive to reward shaping and batching implementation	Higher: Computationally expensive because of complex architectures.
Stability	Lower: Less stable because of sparse rewards	Moderate: Issues with adaptive enhancements, but better stability with Teacher Student framework

Table 5: Comparison of the two RL-based pseudo-labeling approaches

6.5. Practical Considerations

6.5.1. When to Use RL for Pseudo-Labeling

Based on our experience, RL-based pseudo-labeling is most appropriate when labeled data is extremely limited (<1% of dataset), pseudo-labeling decisions have long-term consequences (e.g., medical diagnosis), computational resources are available for extensive training, and the problem exhibits clear reward signals (e.g., validation accuracy).

It may be overkill when labeled data is moderately available (>10% of dataset), simple confidence thresholding already works well, real-time or resource-constrained deployment is required, or the dataset is extremely large (millions of samples).

6.5.2. Engineering Recommendations

From an engineering perspective, several practices proved valuable. Modular design, where environment, agent, and downstream model are separated into distinct modules, facilitated debugging and experimentation. Logging infrastructure, particularly comprehensive logging with TensorBoard, was essential for diagnosing training dynamics and stability issues. Checkpointing provided reliability, as regular model checkpointing allowed recovery from crashes and comparison of models at different training stages. Ablation-friendly code supported rapid experimentation, as designing hyperparameters and enhancement flags in a central config file enabled rapid ablation studies.

6.6. Limitations of Current Work

We acknowledge several limitations. Limited datasets were used, as experiments focused on MNIST and CIFAR-10; generalization to other domains (text, speech, medical imaging) remains unexplored. Hyperparameter sensitivity remains a concern since the limited computational budget prevented extensive hyperparameter searches. Comparison baselines are missing because direct comparisons with strong SSL baselines like FixMatch, MixMatch, or simple confidence thresholding were not performed. Theoretical analysis is another limitation, as no formal convergence guarantees or theoretical characterization of when RL-based pseudo-labeling should outperform alternatives were provided.

7. Conclusion

This capstone project investigated the application of reinforcement learning to intelligent pseudo-labeling in semi-supervised learning contexts. We developed and evaluated two complementary approaches: a custom Policy Gradient framework with explicit state-action formulations, and an implementation of the novel RLGSSL approach with adaptive enhancements.

7.1. Key Contributions

Our work makes several contributions:

1. **Complete RL Framework:** Developed a full-featured custom OpenAI Gymnasium environment for pseudo-labeling, including carefully designed state spaces (image features + model predictions + entropy), action spaces (label assignment or skip), and confidence-weighted reward functions.

2. **RLGSSL Implementation:** Successfully implemented the RLGSSL framework proposed by Heidari et al., including teacher-student architecture, mixup augmentation, and the novel RL reward formulation.

3. **Adaptive Enhancements:** Extended RLGSSL with confidence bonuses, diversity regularization, curriculum learning, and entropy regularization mechanisms not present in the original paper.

4. **Practical Insights:** Documented computational challenges, stability issues, and engineering considerations crucial for practitioners attempting to apply RL to pseudo-labeling.

5. **Open-Source Implementation:** Provided well-documented, modular code for future research and experimentation.

7.2. Main Findings

Technical Feasibility: RL-based pseudo-labeling is technically feasible and can be implemented using standard deep RL techniques. However, significant engineering effort is required to achieve stability and scalability.

Computational Demands: Both approaches exhibit substantial computational requirements, with training times exceeding 15 hours on CPU for moderate-scale experiments. GPU acceleration and algorithmic optimizations are essential for practical deployment.

Stability Challenges: Adaptive reward enhancements, while theoretically motivated, introduced numerical instability. Careful implementation with gradient clipping and reward normalization is crucial.

Performance Gap: Our results showed a significant gap compared to state-of-the-art SSL methods, likely due to incomplete training, implementation bugs, and insufficient hyperparameter

tuning. This highlights the challenge of reproducing research results and the importance of careful validation.

7.3. Lessons Learned

Reward Engineering is Critical: The choice and design of reward functions dramatically affects learning dynamics. Simple confidence-based rewards worked reasonably well, but complex multi-component rewards require careful balancing and stability mechanisms.

Start Simple, Add Complexity Gradually: Our experience suggests starting with minimal RL formulations and incrementally adding enhancements while rigorously testing stability at each stage.

Computational Planning is Essential: RL-based approaches require significantly more computation than standard SSL. Projects should budget computational resources accordingly and prioritize access to GPU infrastructure.

Reproducibility is Challenging: Even with detailed papers, reproducing research results requires careful attention to implementation details, data preprocessing, and hyperparameter settings that may not be fully documented.

7.4. Broader Impact

This work contributes to the growing body of research applying reinforcement learning to data-centric AI problems. As labeled data remains a bottleneck in many domains—especially in medical imaging, autonomous systems, and low-resource languages—intelligent pseudo-labeling strategies could significantly reduce annotation costs and democratize access to machine learning.

However, practitioners should be aware of the computational costs and implementation complexity. For many applications, simpler SSL baselines (FixMatch, MixMatch) may provide better cost-benefit trade-offs. RL-based approaches are most valuable when:

- Labeled data is extremely scarce (<1% of dataset)
- Label quality is critical (high-stakes domains)
- Computational resources are available
- The problem exhibits clear, measurable objectives

7.5. Future Work

To build upon this foundation, future research should focus on stabilizing training dynamics using PPO-style trust region constraints and gradient normalization. Additionally, conducting comprehensive ablation studies to isolate the impact of curriculum learning and confidence bonuses is essential. Finally, scaling the framework via mini-batch RL methods would allow for validation on larger, more complex datasets beyond CIFAR-10.

7.6. Closing Remarks

This capstone project demonstrated both the promise and challenges of applying reinforcement learning to semi-supervised learning. While we encountered significant obstacles—training instability, computational constraints, and performance gaps—the experience provided valuable insights into the complexities of RL-based data-centric AI.

The field of RL-guided semi-supervised learning remains in its early stages, with much room for innovation. As computational resources become more accessible and RL algorithms continue to improve, intelligent pseudo-labeling may become a standard tool in the machine learning practitioner's toolkit.

We hope this work serves as a foundation for future researchers and provides both inspiration and cautionary lessons for those embarking on similar projects.

References

[1] A. Heidari, A. Gheibi, I. Ullah, and N. Salimi, "Reinforcement learning-guided semi-supervised learning," *arXiv preprint arXiv:2405.01760v1*, 2024. [Online]. Available: https://arxiv.org/abs/2405.01760

[2] O. Chapelle, B. Schölkopf, and A. Zien, Eds., *Semi-Supervised Learning*. Cambridge, MA: MIT Press, 2006. [Online]. Available: https://mitpress.mit.edu/9780262033589/semi-supervised-learning/

[3] X. Zhu, "Semi-supervised learning literature survey," Computer Sciences, University of Wisconsin-Madison, Tech. Rep. 1530, 2005. [Online]. Available: https://pages.cs.wisc.edu/~jerryzhu/pub/ssl_survey.pdf

[4] D. Berthelot, N. Carlini, E. D. Cubuk, A. Kurakin, K. Sohn, H. Zhang, and C. Raffel, "Mixmatch: A holistic approach to semi-supervised learning," in *Advances in Neural Information Processing Systems*, vol. 32, 2019. [Online]. Available: https://arxiv.org/abs/1905.02249

[5] K. Sohn, D. Berthelot, N. Carlini, Z. Zhang, H. Zhang, C. A. Raffel, E. D. Cubuk, A. Kurakin, and C.-L. Li, "Fixmatch: Simplifying semi-supervised learning with consistency and confidence," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. [Online]. Available: https://arxiv.org/abs/2001.07685

[6] Q. Xie, M.-T. Luong, E. Hovy, and Q. V. Le, "Unsupervised data augmentation for consistency training," in *Advances in Neural Information Processing Systems*, 2020. [Online]. Available: https://arxiv.org/abs/1904.12848

[7] D. Yarowsky, "Unsupervised word sense disambiguation rivaling supervised methods," in *Proceedings of the 33rd Annual Meeting of the Association for Computational Linguistics*, 1995, pp. 189–196. [Online]. Available: https://aclanthology.org/P95-1026

[8] H. J. Scudder, "Probability of error of some adaptive pattern-recognition machines," *IEEE Transactions on Information Theory*, vol. 11, no. 3, pp. 363–371, 1965. [Online]. Available: https://doi.org/10.1109/TIT.1965.1053799

[9] A. Tarvainen and H. Valpola, "Mean teachers are better role models: Weight-averaged consistency targets improve semi-supervised deep learning results," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. [Online]. Available: https://arxiv.org/abs/1703.01780

[10] H. Zhang, M. Cissé, Y. N. Dauphin, and D. Lopez-Paz, "mixup: Beyond empirical risk minimization," in *International Conference on Learning Representations (ICLR)*, 2018. [Online]. Available: https://arxiv.org/abs/1710.09412

[11] M. Fang, Y. Li, and T. Cohn, "Learning how to active learn: A deep reinforcement learning approach," in *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, 2017, pp. 595–605. [Online]. Available: https://aclanthology.org/D17-1063

[12] S. Narvekar, B. Peng, M. Leonetti, J. Sinapov, M. E. Taylor, and P. Stone, "Curriculum learning for reinforcement learning domains: A framework and survey," *Journal of Machine Learning Research*, vol. 21, no. 181, pp. 1–50, 2020. [Online]. Available: https://jmlr.org/papers/v21/20-212.html

[13] E. D. Cubuk, B. Zoph, D. Mané, V. Vasudevan, and Q. V. Le, "Autoaugment: Learning augmentation strategies from data," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 113–123. [Online]. Available: https://openaccess.thecvf.com/content_CVPR_2019/html/Cubuk_AutoAugment_Learning_Augmentation_Strategies_From_Data_CVPR_2019_paper.html

A. Appendix

A.1. A: Code Structure

The project codebase is organized as follows:

```
Capstone/
 src/
    component/
       main.py           # Main training loop (Policy Gradient)
       model.py          # Policy and Value Networks
       env.py            # Custom PseudoLabelEnv
       utils.py          # Helper functions
       config.py         # Hyperparameter configuration
    requirements.txt      # Python dependencies
 reports/
    RLGSSL_Report_Draft.md     # RLGSSL implementation notes
    Progress_Report/
       Progress_Report.md     # Weekly progress tracking
    Latex_report/
        Final_Report.tex       # This document
        Sample_Report.tex      # Template
        File_Setup.tex         # LaTeX configuration
        references.bib         # Bibliography
        fig/                   # Figures directory
 README.md                      # Project overview
```

A.2. B: Hyperparameter Configuration

Complete hyperparameter settings used in experiments:

Policy Gradient (config.py):

```python
# Training parameters
EPISODES = 50
NUM_LABELED = 1000
NUM_UNLABELED = 4000
NUM_VALIDATION = 1000

# Policy Gradient parameters
GAMMA = 0.99
ALPHA_PI = 1e-3
ALPHA_V = 1e-3
HIDDEN_DIM = 64
POLICY_TYPE = 'mlp'

# Downstream Model parameters
DOWNSTREAM_LR = 0.001
DOWNSTREAM_EPOCHS = 1
BATCH_SIZE = 32
```

RLGSSL (from draft documentation):

```python
# Training Configuration
WARMUP_EPOCHS = 50
RLGSSL_EPOCHS = 400
BATCH_SIZE = 128
LEARNING_RATE = 0.001
```

```
# Teacher-Student
EMA_DECAY = 0.999

# Mixup
MIXUP_ALPHA = 1.0

# Loss Weights
LAMBDA_SUP = 1.0
LAMBDA_CONS = 0.1
LAMBDA_RL = 0.1

# Adaptive Enhancements
CONFIDENCE_THRESHOLD = 0.8
CONFIDENCE_WEIGHT = 0.1
DIVERSITY_WEIGHT = 0.05
ENTROPY_REGULARIZATION = 0.01
```

A.3. C: Key Implementation Details

EMA Teacher Update:

```
def update_ema_teacher(student_model, teacher_model, beta=0.999):
    for teacher_param, student_param in zip(
        teacher_model.parameters(),
        student_model.parameters()
    ):
        teacher_param.data = (
            beta * teacher_param.data +
            (1 - beta) * student_param.data
        )
```

Mixup Generation:

```
def generate_mixup(unlabeled_batch, labeled_batch, alpha=1.0):
    mu = np.random.beta(alpha, alpha)
    x_u, pseudo_y_u = unlabeled_batch
    x_l, y_l = labeled_batch

    # Create mixup samples
    x_m = mu * x_u + (1 - mu) * x_l
    y_m = mu * pseudo_y_u + (1 - mu) * F.one_hot(y_l)

    return x_m, y_m, mu
```

Confidence-Based Reward:

```
def calculate_reward(pred_label, true_label, action, probs):
    if action == num_classes:  # skip action
        return -0.1
    confidence = probs[action].item()
    if action == true_label:
        return confidence  # positive reward for correct
    else:
        return -confidence  # negative reward for incorrect
```

A.4. D: References for Additional Reading

Reinforcement Learning Fundamentals:

- Sutton & Barto (2018), "Reinforcement Learning: An Introduction" (2nd Edition)
- Schulman et al. (2017), "Proximal Policy Optimization Algorithms"
- Mnih et al. (2016), "Asynchronous Methods for Deep Reinforcement Learning"

OpenAI Gymnasium:

- Official Documentation: https://gymnasium.farama.org/
- Custom Environment Tutorial: https://gymnasium.farama.org/tutorials/gymnasium_basics/environment_creation/